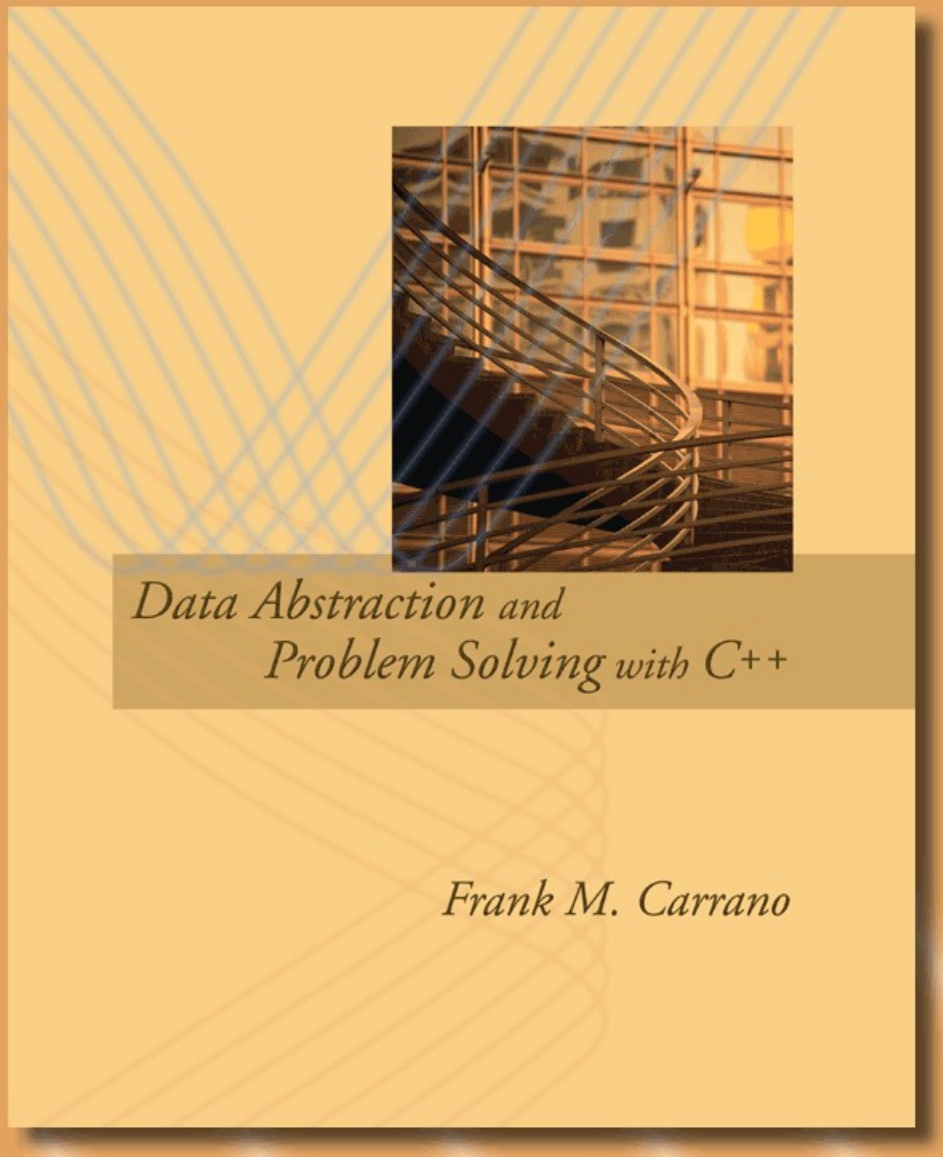
Chapter 6

Stacks

Slides edited by Dr. Selim Aksoy



PEARSON
Addison
Wesley

© 2005 Pearson Addison-Wesley. All rights reserved.

ADT Stack

- A stack
  - Last-in, first-out (LIFO) property
    - The last item placed on the stack will be the first item removed
  - Analogy
    - A stack of dishes in a cafeteria

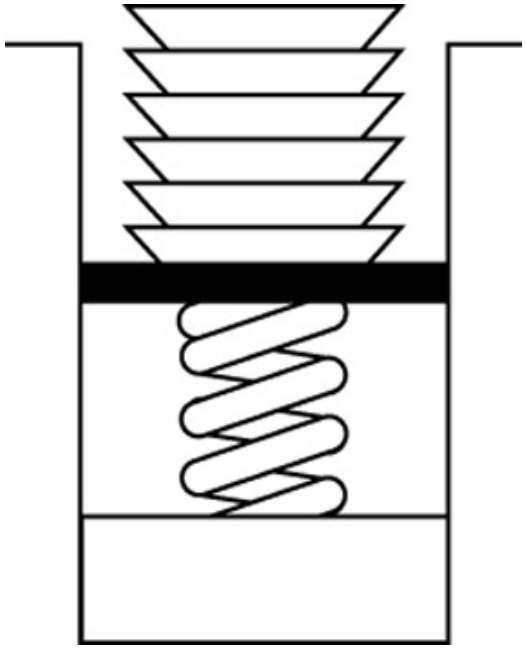


Figure 6.1

Stack of cafeteria dishes

© 2005 Pearson Addison-Wesley. All rights reserved

ADT Stack

- ADT stack operations
  - Create an empty stack
  - Destroy a stack
  - Determine whether a stack is empty
  - Add a new item
  - Remove the item that was added most recently
  - Retrieve the item that was added most recently
- A program can use a stack independently of the stack's implementation

© 2005 Pearson Addison-Wesley. All rights reserved

ADT Stack

```
// stack operations:

   bool isEmpty() const;

   // Determines whether a stack is empty.

   // Precondition: None.

   // Postcondition: Returns true if the
   // stack is empty; otherwise returns false.
```

© 2005 Pearson Addison-Wesley. All rights reserved

ADT Stack

```
bool push(StackItemType newItem);

// Adds an item to the top of a stack.

// Precondition: newItem is the item to
// be added.

// Postcondition: If the insertion is
// successful, newItem is on the top of
// the stack.
```

© 2005 Pearson Addison-Wesley. All rights reserved

ADT Stack

```
bool pop();

// Removes the top of a stack.

// Precondition: None.

// Postcondition: If the stack is not
// empty, the item that was added most
// recently is removed. However, if the
// stack is empty, deletion is impossible.
```

© 2005 Pearson Addison-Wesley. All rights reserved

ADT Stack

```
bool pop(StackItemType& stackTop);

// Retrieves and removes the top of a stack

// Precondition: None.

// Postcondition: If the stack is not empty,
// stackTop contains the item that was added
// most recently and the item is removed.
// However, if the stack is empty, deletion
// is impossible and stackTop is unchanged.
```

© 2005 Pearson Addison-Wesley. All rights reserved

ADT Stack

```
bool getTop(StackItemType& stackTop) const;

// Retrieves the top of a stack.

// Precondition: None.

// Postcondition: If the stack is not empty,
// stackTop contains the item that was added
// most recently. However, if the stack is
// empty, the operation fails and stackTop
// is unchanged. The stack is unchanged.
```

© 2005 Pearson Addison-Wesley. All rights reserved

Checking for Balanced Braces

- A stack can be used to verify whether a program contains balanced braces
  - An example of balanced braces

    ```
    abc{defg{ijk}{l{mn}}op}qr
    ```

  - An example of unbalanced braces

    ```
    abc{def}}{ghij{kl}m
    ```

© 2005 Pearson Addison-Wesley. All rights reserved

Checking for Balanced Braces

- Requirements for balanced braces
  - Each time you encounter a “}”, it matches an already encountered “{”
  - When you reach the end of the string, you have matched each “{”

© 2005 Pearson Addison-Wesley. All rights reserved

Checking for Balanced Braces

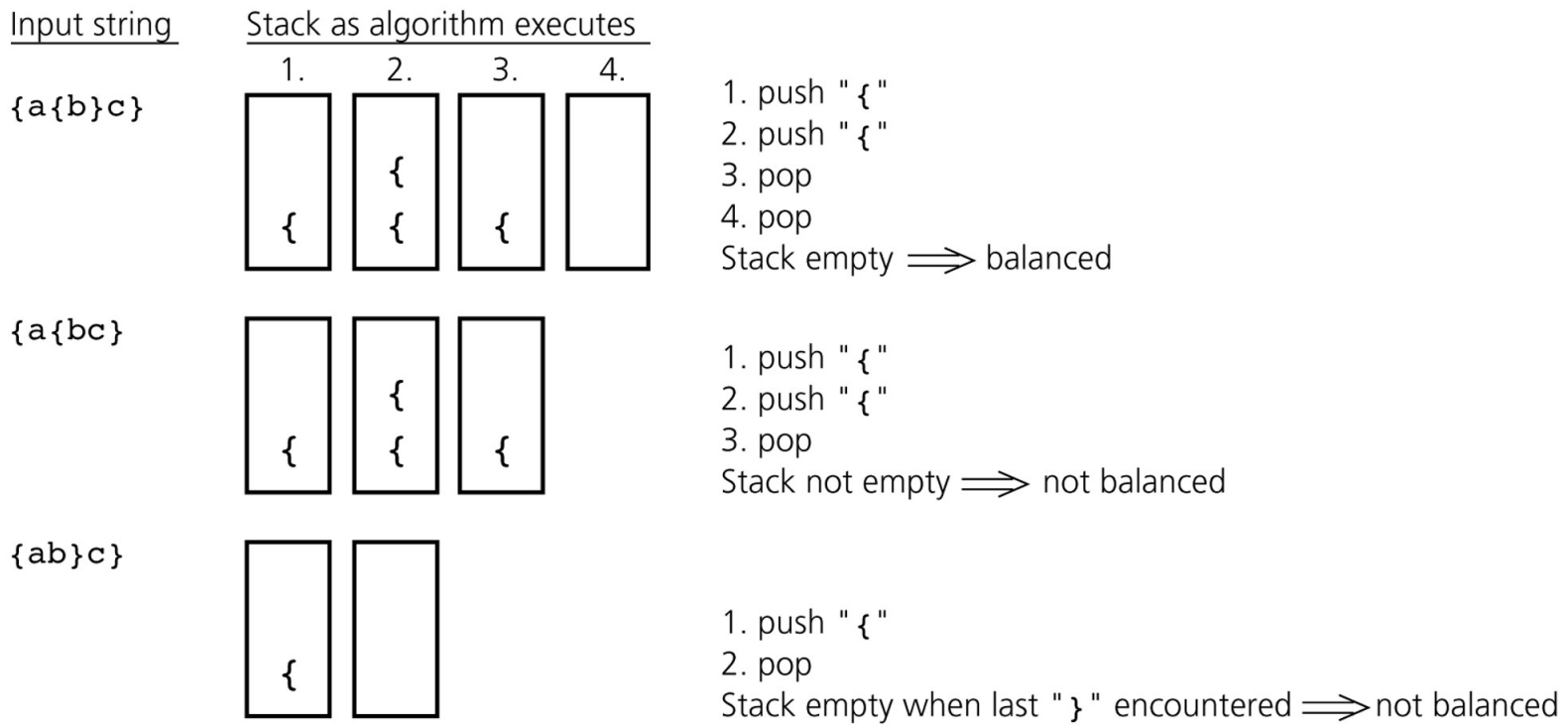


Figure 6.3

Traces of the algorithm that checks for balanced braces

© 2005 Pearson Addison-Wesley. All rights reserved

Checking for Balanced Braces

```
aStack.createStack()
balancedSoFar = true
i = 0

while ( balancedSoFar  and  i < length of aString ){
   ch = character at position i in aString
   ++i

   if ( ch is '{' )       // push an open brace
      aStack.push( '{' )
   else if ( ch is '}' ) // close brace
      if ( !aStack.isEmpty() )
         aStack.pop()    // pop a matching open brace
      else               // no matching open brace
         balancedSoFar = false
      // ignore all characters other than braces
}
if ( balancedSoFar  and  aStack.isEmpty() )
   aString has balanced braces
else
   aString does not have balanced braces
```

© 2005 Pearson Addison-Wesley. All rights reserved

Implementations of the ADT Stack

- The ADT stack can be implemented using
  - An array
  - A linked list
  - The ADT list

© 2005 Pearson Addison-Wesley. All rights reserved

Implementations of the ADT Stack

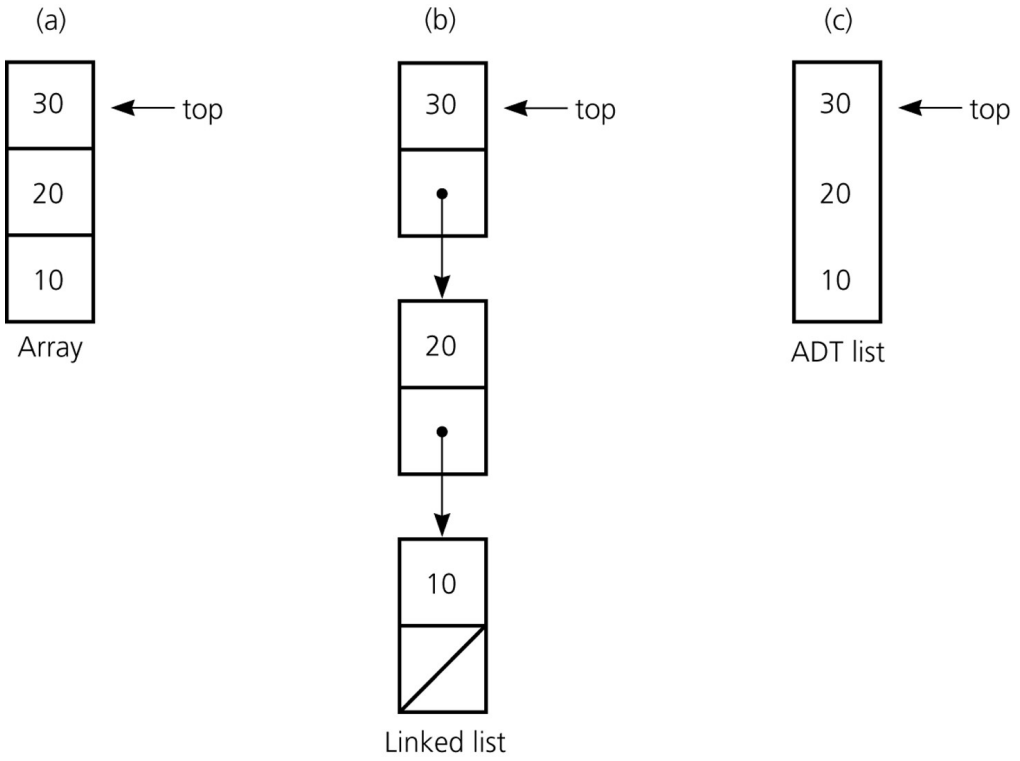


Figure 6.4

Implementation of the ADT stack that use a) an array; b) a linked list; c) an ADT list

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation of the ADT Stack

- Private data fields
  - An array of `items` of type `StackItemType`
  - The index `top`
- Compiler-generated destructor and copy constructor

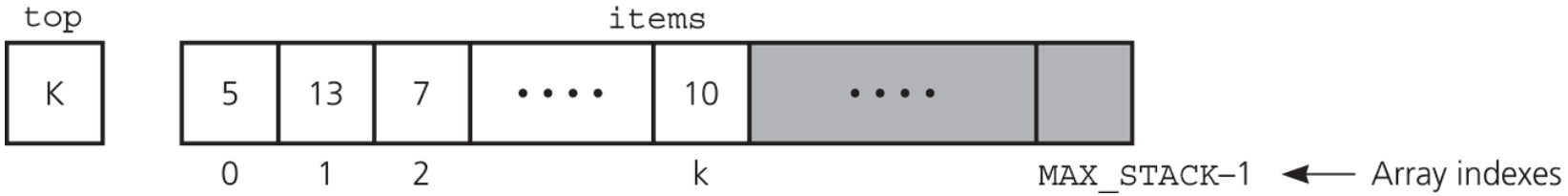


Figure 6.5

An array-based implementation

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation of the ADT Stack

```
const int MAX_STACK = maximum-size-of-stack;
typedef desired-type-of-stack-item StackItemType;

class Stack{

public:
   Stack();

   bool isEmpty() const;
   bool push(StackItemType newItem);
   bool pop();
   bool pop(StackItemType& stackTop);
   bool getTop(StackItemType& stackTop) const;

private:
   // array of stack items
   StackItemType items[MAX_STACK];
   // index to top of stack
   int top;
};
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation of the ADT Stack

```
// default constructor
Stack::Stack(): top(-1){
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation of the ADT Stack

```
bool Stack::isEmpty() const{
    return top < 0;
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation of the ADT Stack

```
bool Stack::push(StackItemType newItem){

    // if stack has no more room for
    // another item
    if (top >= MAX_STACK-1)
        return false;

    else{
        ++top;
        items[top] = newItem;
        return true;
    }

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation of the ADT Stack

```
bool Stack::pop(){

    if (isEmpty())
        return false;

    // stack is not empty; pop top
    else {
        --top;
        return true;
    }

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation of the ADT Stack

```
bool Stack::pop(StackItemType& stackTop){

    if (isEmpty())
        return false;

    // stack is not empty; retrieve top
    else {
        stackTop = items[top];
        --top;
        return true;
    }

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation of the ADT Stack

```
bool Stack::getTop(StackItemType& stackTop)
                const{

    if (isEmpty())
        return false;

    // stack is not empty; retrieve top
    else {
        stackTop = items[top];
        return true;
    }

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT Stack

- A pointer-based implementation
  - Required when the stack needs to grow and shrink dynamically
- `top` is a reference to the head of a linked list of items
- A copy constructor and destructor must be supplied

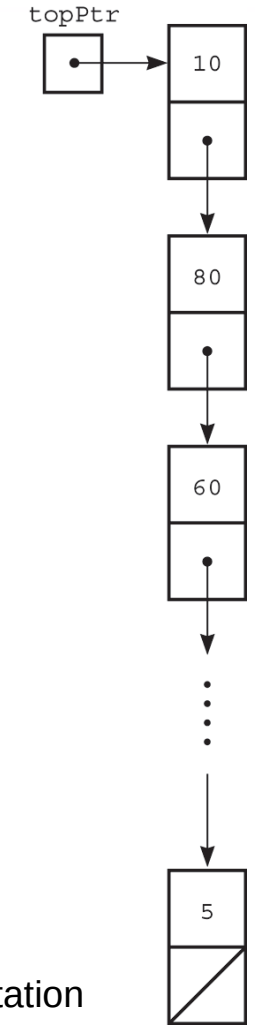


Figure 6.6 A pointer-based implementation

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT Stack

```
typedef desired-type-of-stack-item StackItemType;

class Stack{

public:
   Stack();
   Stack(const Stack& aStack);
   ~Stack();

   bool isEmpty() const;
   bool push(StackItemType newItem);
   bool pop();
   bool pop(StackItemType& stackTop);
   bool getTop(StackItemType& stackTop) const;

private:
   struct StackNode {              // a node on the stack
      StackItemType item;          // a data item on the stack
      StackNode    *next;          // pointer to next node
   };

   StackNode *topPtr;      // pointer to first node in the stack
};
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT Stack

```
// default constructor
Stack::Stack() : topPtr(NULL){

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT Stack

```
// copy constructor
Stack::Stack(const Stack& aStack){

   if (aStack.topPtr == NULL)
      topPtr = NULL;  // original stack is empty

   else {
      // copy first node
      topPtr = new StackNode;
      topPtr->item = aStack.topPtr->item;

      // copy rest of stack
      StackNode *newPtr = topPtr;
      for (StackNode *origPtr = aStack.topPtr->next;
                      origPtr != NULL;
                      origPtr = origPtr->next){
         newPtr->next = new StackNode;
         newPtr = newPtr->next;
         newPtr->item = origPtr->item;
      }
      newPtr->next = NULL;
   }
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT Stack

```
// destructor
Stack::~Stack(){

   // pop until stack is empty
   while (!isEmpty())
      pop();

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT Stack

```
bool Stack::isEmpty() const {

    return topPtr == NULL;

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT Stack

```
bool Stack::push(StackItemType newItem) {

    // create a new node
    StackNode *newPtr = new StackNode;

    // set data portion  of new node
    newPtr->item = newItem;

    // insert the new node
    newPtr->next = topPtr;
    topPtr = newPtr;

    return true;
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT Stack

```
bool Stack::pop() {

    if (isEmpty())
        return false;

    // stack is not empty; delete top
    else{
        StackNode *temp = topPtr;
        topPtr = topPtr->next;

        // return deleted node to system
        temp->next = NULL;  // safeguard
        delete temp;
        return true;
    }
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT Stack

```
bool Stack::pop(StackItemType& stackTop) {

    if (isEmpty())
        return false;

    // not empty; retrieve and delete top
    else{
        stackTop = topPtr->item;
        StackNode *temp = topPtr;
        topPtr = topPtr->next;

        // return deleted node to system
        temp->next = NULL;  // safeguard
        delete temp;
        return true;
    }
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT Stack

```
bool Stack::getTop(StackItemType& stackTop)
                const {

   if (isEmpty())
      return false;

   // stack is not empty; retrieve top
   else {
      stackTop = topPtr->item;
      return true;
   }

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

- The ADT list can be used to represent the items in a stack
- If the item in position 1 is the top
  - `push(newItem)`
    - `insert(1, newItem)`
  - `pop()`
    - `remove(1)`
  - `getTop(stackTop)`
    - `retrieve(1, stackTop)`

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

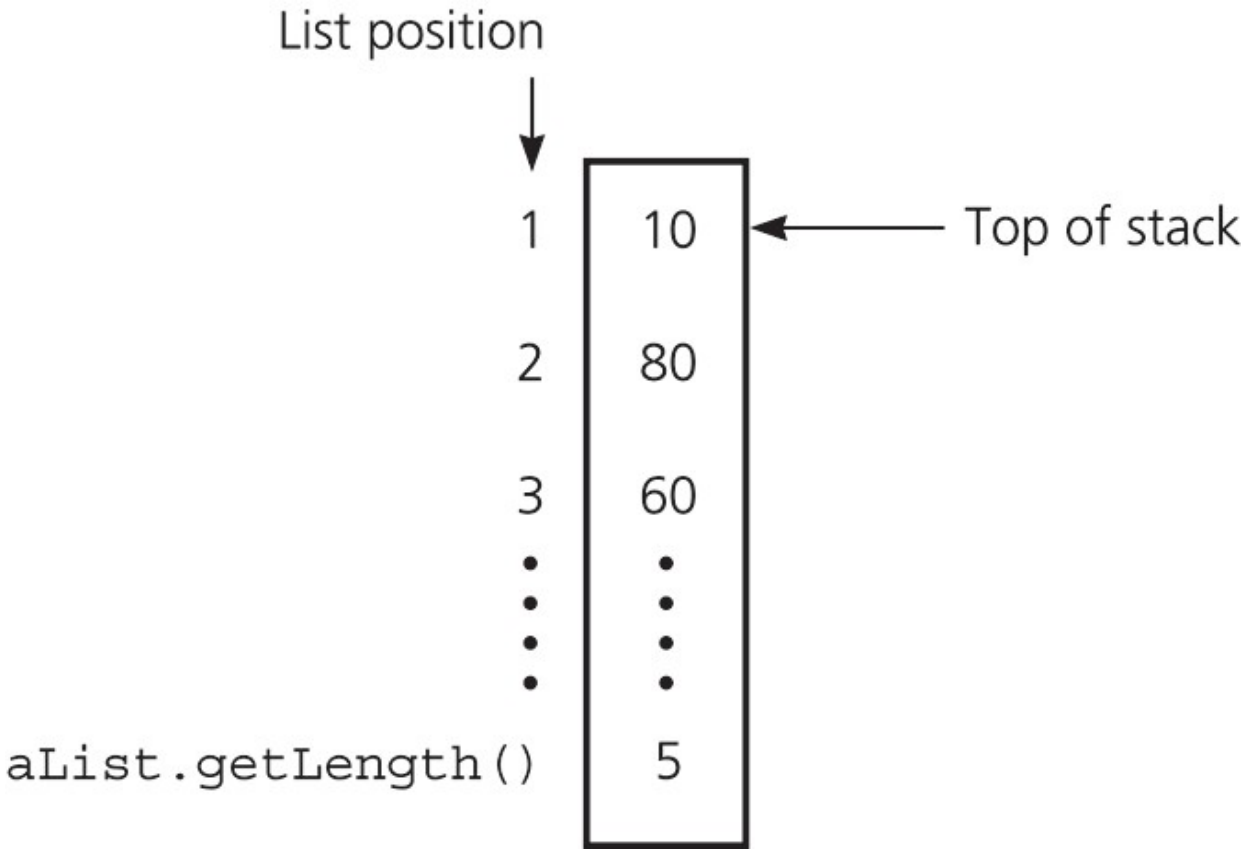


Figure 6.7

An implementation that uses the ADT list

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
#include "ListP.h"      // list operations
typedef ListItemType StackItemType;

class Stack {

public:
   Stack();
   Stack(const Stack& aStack);
   ~Stack();

   bool isEmpty() const;
   bool push(StackItemType newItem);
   bool pop();
   bool pop(StackItemType& stackTop);
   bool getTop(StackItemType& stackTop) const;

private:

   List aList;  // list of stack items

};
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
// default constructor
Stack::Stack(){

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
// copy constructor
Stack::Stack(const Stack& aStack)
            : aList(aStack.aList){

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
// destructor
Stack::~Stack() {

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
bool Stack::isEmpty() const {

   return aList.isEmpty();

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
bool Stack::push(StackItemType newItem){
     return aList.insert(1, newItem);
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
bool Stack::pop() {
    return aList.remove(1);
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
bool Stack::pop(StackItemType& stackTop) {

    if (aList.retrieve(1, stackTop))
       return aList.remove(1);
    else
       return false;

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
bool Stack::getTop(StackItemType& stackTop)
        const {

   return aList.retrieve(1, stackTop);

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

Comparing Implementations

- Fixed size versus dynamic size
  - An array-based implementation
    - Prevents the `push` operation from adding an item to the stack if the stack's size limit has been reached
  - A pointer-based implementation
    - Does not put a limit on the size of the stack

© 2005 Pearson Addison-Wesley. All rights reserved

Comparing Implementations

- An implementation that uses a linked list versus one that uses a pointer-based implementation of the ADT list
  - ADT list approach reuses an already implemented class
    - Much simpler to write
    - Saves time

© 2005 Pearson Addison-Wesley. All rights reserved